Rapport TP : Optimisation d'un service de livraison

Gestion des colis

Le tableau dynamique est choisi comme stockage principal car il permet de conserver les colis de façon compacte, de les parcourir efficacement et d'augmenter sa capacité lorsque nécessaire. La recherche d'un colis par identifiant est réalisée par un parcours séquentiel du tableau. Cette solution reste simple à implémenter, mais son coût augmente linéairement avec le nombre de colis.

Opération	Stratégie	Temps	Espace	Justification
Ajouter	Tableau dynamique, doublement de capacité si plein	O(n)	O(n)	Le redimensionnement est amorti, mais la vérification d'un identifiant déjà présent parcourt le tableau.
Afficher tous les colis	Parcours séquentiel	O(n)	O(1)	On doit de toute façon lire les n colis affichés.
Rechercher par ID	Parcours séquentiel	O(n)	O(1)	Simple et adapté à une collection modérée.
Afficher par poids, valeur ou priorité	Copie temporaire des structures Colis puis tri rapide	O(n log n)	O(n)	La collection principale n'est pas modifiée : seul l'affichage est trié.

Le tri rapide a été choisi car il présente une complexité moyenne en O(n log n) et utilise peu de mémoire supplémentaire. Il est plus adapté que les tris quadratiques pour une collection allant jusqu'à 100 000 colis. Son pire cas est en $O(n^2)$ lorsque les partitions sont déséquilibrées. Pour réduire ce risque, le pivot est choisi au milieu du tableau.

Tri	Meilleur cas	Cas moyen	Pire cas	Mémoire	Choix
À bulles optimisé	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Non, trop lent à 100 000 colis
Par sélection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Non, trop lent
Par insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Non, sauf tableau presque déjà trié
Fusion	O(n log n)	O(n log n)	O(n log n)	O(n)	Très fiable, mais plus complexe à coder
Rapide	O(n log n)	O(n log n)	$O(n^2)$	Faible, via récursion	Oui, bon compromis

Préparation du camion

La programmation dynamique a été choisie car le problème correspond à un sac à dos 0/1 : chaque colis est soit chargé une fois, soit ignoré. La table mémorise la meilleure valeur possible pour chaque nombre de colis et chaque capacité. Cela évite de recalculer plusieurs fois les mêmes sous-problèmes.

Pour chaque colis et chaque capacité, l'algorithme compare systématiquement les deux possibilités : ne pas prendre le colis ou le prendre s'il tient dans le camion. La meilleure des deux valeurs est conservée. La dernière case de la table correspond donc à la meilleure valeur totale possible sans dépasser la capacité.

La complexité temporelle est O(n × C), où n est le nombre de colis et C la capacité maximale du camion. Chaque case de la table est calculée une seule fois.

La complexité spatiale est O(n × C), car une table contenant n + 1 lignes et C + 1 colonnes est stockée en mémoire.

Une stratégie gloutonne pourrait trier les colis selon leur valeur, leur poids ou leur rapport valeur/poids. Elle serait plus rapide, mais ne garantit pas une solution optimale. Une recherche exhaustive testerait toutes les combinaisons, mais son coût en O(2^n) deviendrait rapidement trop important.

Prise en compte des priorités

Pour sélectionner les colis, le poids est traité comme une contrainte stricte : le poids total du chargement ne doit jamais dépasser la capacité maximale du camion.

Parmi les combinaisons respectant cette contrainte, un score est calculé pour chaque colis :

score = valeur + (10 × priorité)

La valeur représente l'intérêt économique du colis. La priorité ajoute un bonus de 10 points par niveau : un colis de priorité élevée devient donc plus intéressant, même si sa valeur brute est plus faible.

La programmation dynamique cherche alors la combinaison dont le score total est le plus élevé, sans dépasser la capacité du camion. Cette règle permet de prendre simultanément en compte le poids, la valeur et la priorité.

Le coefficient 10 a été retenu car les valeurs des colis de l'exemple sont comprises entre 10 et 40. La priorité influence donc réellement la sélection, sans remplacer complètement l'importance de la valeur économique.

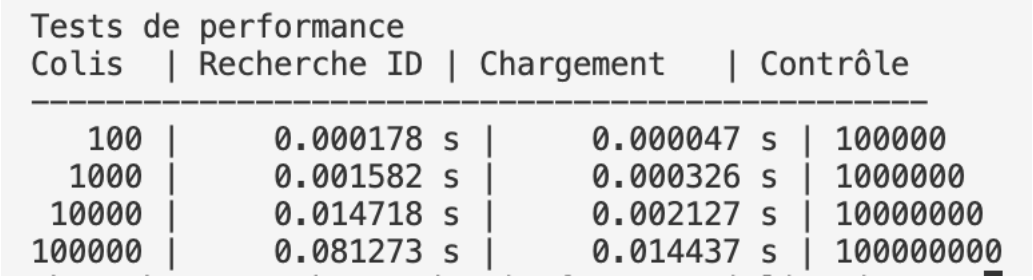
Recherche et performance

Opération	**Complexité**	**Évolution avec n**	**Adaptée à 100 000 colis ?**	**Amélioration**
Recherche d'un colis par ID	O(n)	Le temps augmente linéairement lorsque le nombre de colis augmente.	Correcte pour peu de recherches, moins adaptée avec 100 000 colis et des recherches fréquentes.	Utiliser une table de hachage pour atteindre O(1) en moyenne.
Préparation du chargement	O(n × C)	Le temps augmente avec le nombre de colis n et avec la capacité C du camion. Ici C = 50, donc l'évolution reste raisonnable.	Oui, pour les capacités testées.	Utiliser une table réduite à une dimension pour diminuer la mémoire.

La recherche d'un colis par identifiant est actuellement réalisée par un parcours séquentiel du tableau. Sa complexité est donc O(n). Lorsque le nombre de colis augmente, le temps de recherche augmente linéairement : une collection de 100 000 colis peut nécessiter jusqu'à 100 000 comparaisons dans le pire cas. Cette solution reste simple et adaptée pour peu de recherches, mais elle devient moins efficace si les recherches sont fréquentes. Une table de hachage associant chaque identifiant à la position du colis permettrait une recherche moyenne en O(1).

L'affichage des colis triés utilise un Quick Sort. Sa complexité moyenne est O(n log n), ce qui reste adapté à 100 000 colis. Cependant, dans un cas défavorable, le Quick Sort peut atteindre $O(n^2)$. Le pivot placé

au milieu limite ce risque pour des données déjà triées. Une autre amélioration possible serait le tri fusion, qui garantit une complexité de O(n log n) dans tous les cas, mais utilise davantage de mémoire.



```
Tests de performance
Colis  | Recherche ID | Chargement   | Contrôle
-----------------------------------------------
   100 |     0.000178 s |     0.000047 s | 100000
  1000 |     0.001582 s |     0.000326 s | 1000000
 10000 |     0.014718 s |     0.002127 s | 10000000
100000 |     0.081273 s |     0.014437 s | 100000000
```

La préparation du chargement repose sur une programmation dynamique. Avec une capacité fixée à 50 kg, son coût évolue linéairement avec le nombre de colis : O(n × 50), donc pratiquement O(n). Cette solution reste adaptée aux tailles testées. En revanche, si la capacité maximale du camion devenait très grande, le coût augmenterait fortement ; une optimisation possible serait de réduire la mémoire utilisée par la table dynamique.